FraudArena: ready first, weights later

Implementation plan v2 for the team (M1 to M4). Replaces the earlier PDF. Budget: 5 hours. Companion to docs/PLAN_RBI_ALIGNMENT.md.

1. The one rule that shapes everything

All code is finished, merged and tested before training ends. When the trained weights arrive, one command installs them and every part of the product updates by itself. No code is edited after training. If training finishes late, or never, the product still works in 'Simulation' mode and says so honestly.

- **Why:** the judge doubts our simulated data and wants plain, RBI-mapped reasons for every decision. The RBIH Phase-2 data, trained on Kaggle by a teammate's friend, is the data-backed answer; the rules and UI answer the 'understandable' objection.
- **How:** everything that depends on the data reads from one place, a versioned **artifact bundle** (section 3), through one resolver, with a defined fallback for every missing piece.
- **Proof it works before training ends:** a **rehearsal bundle** is built from the fake fixture (kaggle/make_fixture.py plus the real Kaggle exporter). Every test, the UI and the install command are exercised on it. The real bundle has the same shape, so installing it is a non-event.

2. Where we are

- **Done and pushed (e7bef9d):** the Kaggle pipeline in kaggle/ (SQL features proven equal to fa/core/features.py, leakage guards, stratified account split, XGBoost plus depth-4 tree, aggregates-only export, fake-data dry run, tests).
- **Gap found (must fix first):** the exporter does not yet write schema_version, a rehearsal flag, a list of unavailable features, or a deck-numbers summary. The app needs these. This is task K-1 below and must land **before your friend starts stage 2**.
- **Waiting on:** the friend's stage-1 report (schema) and the stage-2 bundle. The app code does not wait for either.

3. The artifact bundle (the contract between Kaggle and the app)

One folder, installed to data/rbih/current/. Defined once in fa/rbih/bundle.py (pydantic models) and checked by a contract test that runs the real Kaggle exporter on the fixture and loads the result with the app loader. If either side drifts, the test fails.

File	Required?	Used by	If missing or invalid
model_card.json (schema_version, created_at, rehearsal, dataset text, sha256 of every file, unavailable_features, gpu_used, limits)	Required	Installer, badge, docs	Install refused
model.json (XGBoost)	Required for scoring	Scorer: score and model reasons	No score or model reasons; rules and rule reasons still work
friction_tables.json (per feature: threshold ladder, legit and mule flagged % on train+val and on test)	Required for data-backed thresholds	Threshold resolver, Policy page evidence	Thresholds fall back to params.yaml and are labelled 'simulation'
metrics.json (PR-AUC, recall at 0.5, 1, 2% friction, per alert reason if present)	Required	Model panel, RESULTS_RBIH.md, deck numbers	Install refused
mined_rules.json	Optional	Defender candidates (still pass G1 to G6)	Skipped; may be empty
feature_spec.json (feature order, units, notes)	Required	Scorer input check	Install refused if order differs from FEATURES
schema_used.json (column mapping, warnings)	Optional	Data card	Data card says 'mapping not recorded'

4. Install command and live update

python -m fa.rbih.install path/to/rbih_artifacts.zip (also accepts a folder; --dry-run checks only; --rehearsal installs a rehearsal bundle; --rollback restores the previous one). Steps, in order, stopping at the first failure and leaving the current bundle untouched:

- Safe unzip (reject path traversal), verify every sha256 against model_card.json, check schema_version and that feature order equals FEATURES.
- Gates: rehearsal flag must be false for 'real' mode; PR-AUC or ROC-AUC above 0.99 needs --accept-high-auc after a leakage review; each friction-table rung used must be within budget on the test split; the model loads and gives a deterministic score on a probe row.
- Derive thresholds by the recorded procedure (loosest rung within the rule's friction budget, confirmed on test) and write derived_thresholds.json with evidence.
- Generate docs/RESULTS_RBIH.md and docs/deck_numbers.json from metrics.json (numbers come only from here, never typed by hand).
- Swap atomically into data/rbih/current/, keep the old bundle in data/rbih/previous/, and append a **ledger entry** (model_installed, bundle sha256, installer, mode). This doubles as an audit trail.
- Tell the running API to reload. The API also checks the bundle's modified time on each /api/model request, so even a manual copy is picked up without a restart.

Mode	Meaning	Badge	Thresholds	Docs may quote numbers?
none	No bundle installed	Simulation	params.yaml	Simulation numbers only, labelled
rehearsal	Bundle built from fake data	Simulation (rehearsal)	params.yaml (never derived from fake data)	No
real	Verified RBIH-trained bundle	Data: RBIH NFPC Phase-2 (derived)	derived_thresholds.json	Yes, from metrics.json only

5. What updates by itself when weights arrive (and what it does if they are absent)

Consumer	Reads	Automatic effect	Fallback
Policy pack thresholds (fa/rules)	derived_thresholds.json via one resolver	Rule thresholds and evidence become data-backed; Policy page shows rung and friction	params.yaml values, labelled simulation
Rules needing unavailable features	model_card.unavailable_features and each rule's requires list	R4 (device) is shown as 'not evaluable on this data', not claimed as data-backed	Normal behaviour in simulation
/tx and /decisions (api)	scorer + explain()	Score and model reasons added next to rule reasons	Rule reasons only
Defender candidates (fa/ml/mine_rules)	mined_rules.json	Extra candidates, same scoring and G1 to G6	Existing candidate search
Header badge, Model panel (web)	GET /api/model	Badge, metrics at 0.5, 1, 2% friction, dataset text, hashes verified	'Simulation'
Docs and deck	RESULTS_RBIH.md, deck_numbers.json	Regenerated numbers	Not generated
Ledger	install event	model_installed entry appears and verifies	-

6. Rules every deliverable must follow

- **Provenance:** one third party calls the dataset synthetic data based on real mule patterns; the challenge site does not say. Quote the Kaggle text verbatim (stored in model_card.dataset and shown by the app). Say 'RBIH-published', never 'real bank data'.
- **Licence:** reported CC BY-NC-SA 4.0 (unconfirmed). Non-commercial, share-alike, attribution. No raw rows in git or in this repo; only aggregates and the model, and only after the friend confirms the terms allow it.
- **Limits stated everywhere:** no device field; labels are account-level; attacks are simulated; G5 keeps using simulated normal traffic because RBIH rows never reach this repo.
- **Regulatory wording:** 'supports' or 'aligned with', never 'compliant'. Strength labels: Mandated, Permitted, Industry, Best practice, Unverified. Do not cite Unverified items.
- **Defensive reading:** every optional bundle field may be absent or empty (for example no mined rules, no per-alert-reason metrics). Missing means fall back, never crash.
- **Repo rules still apply:** one owner per file, money in integer paise, thresholds in params.yaml or derived files (no magic numbers), [Mx] commit prefixes, pull --rebase before push, no force push.

7. Work packages

Owners follow docs/TIMELINE.md. Ask the file's owner before editing it. Everything below is done and tested on the rehearsal bundle; nothing waits for real weights.

M3 - bundle, installer, scorer (critical path)

ID	Task	Files	Done when
K-1 (first, before stage 2)	Patch the Kaggle exporter to write schema_version, rehearsal (false by default; true when run on the fixture), unavailable_features (device_changed_24h on this data), dataset text from CFG, and a manifest. Re-run the dry run; push; tell the friend to pull before stage 2.	kaggle/rbih_export.py, rbih_config.py, tests/test_rbih_kaggle.py	Dry-run card has all fields; friend has the new commit
M3-B	bundle.py: pydantic models for the contract and a verify() that checks hashes, schema version, feature order. Contract test: real exporter on the fixture, then app loader.	fa/rbih/bundle.py (new), tests/test_rbih_contract.py	Drift on either side fails a test
M3-C	Installer: safe unzip, gates, derived thresholds, generated docs, atomic swap, rollback, ledger entry (use fa.ledger.chain.append). Includes --dry-run, --rehearsal, --rollback.	fa/rbih/install.py (new)	Install, re-install and rollback pass on the rehearsal bundle; a tampered zip is refused
M3-D	Scorer: xgboost model.json plus pred_contribs gives score and top-3 reasons in plain text; CPU, deterministic; returns None when no model.	fa/rbih/scorer.py (new)	Test with rehearsal model; absent model returns None
M3-E	Resolver: one function gives the effective threshold and its evidence for a feature (derived if present and valid, else params). Used by the policy pack and candidate search.	fa/rbih/resolver.py (new), fa/ml/mine_rules.py (candidates and ladder lookup)	Switching the bundle changes thresholds with no code change (test)
M3-F	Importer for user data (CSV or Parquet: event_id, ts, type, account, payee, amount, optional device) and python -m fa.metrics --events file. Data card and a generated RESULTS_RBIH.md template.	fa/world/importer.py, docs/DATA_CARD.md	Round-trip test

M4 - rules and verifier

ID	Task	Files	Done when
M4-A	explain(rule): plain title, customer message, analyst note from a feature glossary (mirror kaggle/rbih_explain.py).	fa/rules/explain.py (new)	Fuzz test: no raw feature names in output
M4-B	Policy pack R1 to R7 (section 8) with a requires list per rule; thresholds come from the resolver. One source of truth: delete the duplicate root baseline_v1.json and the stale fallback copy.	fa/rules/policy_pack.py (new), fa/rules/loader.py	load('v1') gives 7 rules with text fields; rules with unavailable features are flagged

ID	Task	Files	Done when
M4-C	Update tests that pin R1 to R6 (section 11). Re-check friction <= 1%, all 200 golden cases allowed, attacker loss re-measured.	tests listed in section 11	make test green
M4-D	docs/REGULATORY_MAP.md (section 9) with sources and strength labels; confirm clause numbers in the primary texts.	docs/REGULATORY_MAP.md	Every row sourced and labelled

M1 - contracts, agents, config

ID	Task	Files	Done when
M1-A	Optional fields on Rule: title, customer_message, analyst_note, treatment (str) and reg_refs (list[dict]), all defaulted. Announce: frozen file, backward compatible. Do this first (hour 0:30).	fa/contracts.py	Existing tests green
M1-B	Defender fills the text fields from explain(); the LLM never writes customer-facing text.	fa/agents/defender.py	Defender-built rule has the fields
M1-C	params.yaml: remove the duplicated ml.export and mine_rules blocks (about lines 386 to 403); add rbih: (enabled, dir, budgets). Makefile targets: rbih-install, rbih-rehearsal.	params.yaml, Makefile	One ml.export; targets run

M2 - API and dashboard

ID	Task	Files	Done when
M2-A	/tx returns reasons (rule titles and customer messages) and, if a scorer exists, score and model reasons; store them inside the features JSON. api/main.py loads rules through the loader.	api/routes_core.py, api/main.py	curl shows plain reasons
M2-B	GET /api/model (mode, dataset text, hashes_verified, metrics summary, unavailable features) and POST /api/model/reload; mtime check on each request.	api/routes_core.py or routes_rounds.py	Returns 'none' until a bundle is installed
M2-C	Header badge and a Model panel (recall at 0.5, 1, 2% friction, dataset text, limits). Policy page (Rulebook). Arena rules table and Patch page show plain text and regulatory chips (Patch.tsx is M4's: agree first).	web/src/views/Policy.tsx (new), Arena.tsx, Patch.tsx, App.tsx, hooks/useArena.ts	Badge flips live after an install, both themes
M2-D	Screenshots for the deck (both themes, simulation and rehearsal).	-	Images ready
M2-E	'How it works' page for non-technical readers (section 13). **First version is built** (web/src/views/HowItWorks.tsx, nav entry, route, styles). Remaining: make the 'Everyday behaviour' paragraph follow GET /api/model (none, rehearsal, real) so it never overclaims; owners review the wording.	web/src/views/HowItWorks.tsx, web/src/App.tsx, styles/theme.css	Page text matches the badge in all three modes; plain-language review passed

8. Policy pack draft (use this wording; thresholds come from the resolver)

Rule	Plain title and customer message	Condition (feature keys)	Treatment	Regulatory anchor and strength
R1	**Much larger than usual.** This payment is much larger than your usual payments, so we paused it to check it is you.	amount_ratio_30d >= T1	Hold + step-up authentication	RBI Authentication Directions 2025, risk-based extra checks (Permitted). RBI Digital Payment Security Controls 2021, fraud risk management (Mandated, chapter level).
R2	**New payee, big amount.** You added this person very recently and the amount is large. Please confirm it is really them.	payee_age_h < T2 and amount_ratio_30d >= T2b	Hold + confirmation or callback	Bank cooling-off practice (Industry). Do not cite the unverified Rs 2,000 new-payee claim.
R3	**Too many payments in an hour.** We saw an unusual burst of payments and stopped it for your safety.	n_pay_1h >= T3	Block	Digital Payment Security Controls 2021, fraud risk management (Mandated, chapter level; confirm the velocity clause).
R4	**New phone, money moving.** This payment came from a phone we have not seen before.	device_changed_24h == 1 and sum_1h >= T4	Hold + step-up authentication	Authentication Directions 2025: device data as a risk factor (Permitted). requires: device (not in RBIH data, so shown as not evaluable there).
R5	**Paying many different people quickly.** Money was sent to many different people in a short time.	fan_out_1h >= T5	Block	RBIH MuleHunter.AI patterns (Best practice). Fraud Risk Management Master Directions 2024, early warning (Mandated, governance).
R6	**Large payment at night.** A large payment was attempted late at night.	night_flag == 1 and amount_ratio_30d >= T6	Hold + step-up authentication	Authentication Directions 2025: contextual parameters such as time (Permitted).
R7 (new)	**Money passes straight through.** This account receives from many senders and sends the money straight out.	r_fan_in_24h >= T7a and r_passthrough_1h >= T7b	Hold + review	RBIH MuleHunter.AI (Best practice). Fraud Risk Management Master Directions 2024 (Mandated, governance).

'Hold' has no release flow in the product today. Describe it as a treatment in the text; do not build a fake release flow.

9. Regulatory anchors (web-checked; confirm clause numbers in the primary texts before quoting)

Instrument	What it gives us	Strength
RBI Master Direction on Digital Payment Security Controls, 18 Feb 2021 (RBI/2020-21/74)	Chapters on authentication, fraud risk management, customer protection	Mandated (chapter level)
RBI Authentication Mechanisms for Digital Payment Transactions Directions, 2025 (25 Sep 2025, effective 1 Apr 2026)	2FA baseline; issuers may add risk-based checks using device, history, location	Permitted
RBI Master Directions on Fraud Risk Management, 15 Jul 2024 (RBI/DOS/2024-25/118)	Board-approved policy, early-warning framework; the bank defines its indicators	Mandated (governance)
RBI Customer Protection - Limiting Liability, 6 Jul 2017 (RBI/2017-18/15)	SMS alerts, 24x7 reporting, zero liability if reported within 3 working days	Mandated
RBIH MuleHunter.AI (2024)	ML mule detection using 19 mule behaviour patterns, near real time	Best practice
NPCI/UPI limits from bank FAQs: Rs 1 lakh per transaction; Rs 5,000 in first 24h for newly registered users	Context only; enforced by the payment rail	Industry
'First payment above Rs 2,000 to a new payee is held'	Found on one website only	**Unverified: do not cite**

10. Schedule (code complete first, install any time)

Hour	Team	Friend (Kaggle)
0:00 to 0:30	M3: K-1 exporter patch and push. M1-A contracts. M3 sends kaggle/ and the checklist.	Creates the Kaggle dataset, accepts rules, runs stage 1 (about 10 minutes)

Hour	Team	Friend (Kaggle)
0:30 to 1:15	M3-B contract, M4-A explain(), M1-B/C.	Sends schema_report.json and licence text. **Checkpoint 1: mapping confirmed.** Pulls the K-1 commit, starts stage 2
1:15 to 3:00	M3-C installer, M3-D scorer, M3-E resolver, M4-B policy pack, M2-A/B.	Training runs
3:00 to 3:30	M2-C UI, M3-F importer, M4-C tests. **Code freeze at 3:30: everything merged and green on the rehearsal bundle.**	Training runs
3:30 to 4:30	Fire drill: install rehearsal bundle, re-install, tamper test, rollback, screenshots (M2-D); docs and deck bullets; M4-D regulatory map.	Artifacts due
4:30 to 5:00	**Install day:** run the runbook below, check the badge, numbers and ledger; final tests; push.	Sanity-checks metrics with us

If the real bundle arrives early or late, only the install step moves. After code freeze nothing else needs a code change. Cut order if late: importer, then Policy page polish, then the Model panel.

Install-day runbook (5 to 10 minutes, no code edits)

- Put rbih_artifacts.zip next to the repo. Run **python -m fa.rbih.install rbih_artifacts.zip --dry-run**. Read the gate report.
- If clean, run it without --dry-run. If a gate trips (for example AUC above 0.99), stop: ask the friend to re-check the label window; do not override.
- Open the dashboard: badge reads 'Data: RBIH NFPC Phase-2 (derived)'; Policy page shows data-backed evidence; Model panel shows recall at 0.5, 1, 2% friction; Ledger shows model_installed and verifies.
- Run python -m pytest -q and python -m fa.metrics --split validation. Commit the generated docs (RESULTS_RBIH.md, deck_numbers.json) and, only if the friend confirms the terms allow it, the small aggregate files.
- Something wrong: **python -m fa.rbih.install --rollback** restores the previous bundle.

11. Tests and definition of done

Existing tests that pin R1 to R6 (update with each file's owner):

- tests/test_engine.py (ids R1..R6, equality with baseline_rules_from_params(), actions, R1 fires at 30k vs 1k average, first block at the 13th payment); tests/test_gates.py (duplicate of R1 message, baseline does not stop the attack); tests/test_release.py (rule count 6, ids R1..R6 plus R-night); tests/test_ml.py (200 golden cases allowed by v1; baseline ids); tests/test_features.py (R2-only band); tests/test_plans.py (fan-out of 9 blocked by R5); tests/test_agents.py (loss against v1); tests/test_api.py (version v1); tests/test_world.py (friction <= 2%); tests/test_metrics.py (experiment on v1).

New tests: contract (exporter on fixture to app loader); installer (install, re-install, tampered zip refused, rehearsal not shown as real, rollback, ledger entry); resolver switches thresholds with the bundle; scorer returns reasons and None when absent; explain fuzz; policy pack matches resolver; importer round-trip.

Definition of done:

- Full suite green with the rehearsal bundle built by a session fixture; v1 friction <= 1%; all golden cases allowed.
- Installing a different bundle changes thresholds, badge, Model panel, generated docs and ledger with **zero code changes**; rollback restores the previous state.
- No raw feature names visible to users; plain reasons from /tx; Policy page and badge work in both themes.
- The 'How it works' page is understandable to a non-technical reader and its data statement matches the badge in all three modes.
- Docs quote numbers only from generated files; provenance and licence wording comes from model_card.dataset; the deck says 'supports/aligned with'.

12. Contingencies

If	Then
Stage 1 shows column auto-detect failed	Friend sets CFG.txn_columns / label_columns; M3 adjusts amount unit and debit/credit values in kaggle/rbih_config.py (about 10 minutes); no app change.
No counterparty or direction column	Payee-based features degrade; model_card.unavailable_features lists them; affected rules shown as not evaluable; no data-backed claim for them.
metrics.json shows PR-AUC or ROC above 0.99	Installer refuses without --accept-high-auc; review the label window and flag-date handling with the friend first.
Recall at 1% friction is low	Report it honestly; the model is a second line, the rules the auditable first line; never tune on test.
Mined rules fail G1 to G6 or the file is empty	Skip them; thresholds still come from friction tables.
No bundle arrives	Ship in Simulation mode (or rehearsal, clearly labelled); the deck says the RBIH run is in progress and shows the proven parity test and the install fire drill. Never present fake numbers as real.

13. Workflow explainer page ('How it works')

Why: a judge said he did not understand how the project works. This page explains the whole loop to a normal user in plain words and shows what it means for them. Route /how-it-works, last item in the top navigation. It is static text, so it works offline and never calls the API (except the planned mode-aware paragraph below).

Section on the page	What it says (no jargon)	Why it is there
Idea in one sentence + 'What it means for you'	Banks crash-test their fraud rules with pretend attacks, in a safe place, before real customers are on the road. Three promises: scammers stopped sooner, honest payments left alone, you always get a reason.	Answers 'what is this?' in 10 seconds
The loop, step by step (6 steps)	1 A pretend criminal tries to steal. 2 We measure what got through. 3 A defender suggests one fix. 4 Six safety checks. 5 A second person approves. 6 It goes live and is written in a tamper-proof diary. Each step has a 'Why it matters' line.	The actual workflow, readable by anyone
Six safety checks in plain words	G1 to G6 translated: well formed; stops the attack; does not bring back old problems; survives small changes; does not bother normal customers; honest-but-unusual payments still go through.	Shows safety without technical terms
Try it: what would you see?	Three example payments (usual shop payment, large payment to a brand-new person, burst of 14 payments in an hour) show Allowed, Paused for a check, or Stopped, with the customer message, the reason and what happens next. Marked as an illustration.	Makes the customer experience and 'reason for every decision' concrete
Why this is good for customers	Fewer scams get through; fewer false alarms (capped share of normal payments); every decision has a reason; people, not robots, decide; mistakes can be undone; everything can be audited.	The benefit case for end users
What is real and what is practice	Attacks are simulated (cannot be replayed safely). Everyday behaviour: simulated today, RBIH public challenge data being added. Rules and approval trail are real. Design alignment with RBI expectations, not a compliance certificate.	Honest about the data; answers the 'virtual data' doubt
Common questions, words you will see	Is real money at risk? Does AI decide? Can it be wrong? Why attack your own rules? Plus a six-word glossary (rule, pause, stop, friction, ledger, second approver).	Removes the remaining confusion

Acceptance: readable by someone with no banking or technical background (read it aloud to a non-engineer); no raw feature names or gate codes without a plain explanation; works in both themes and at phone width; **the data paragraph follows the installed bundle**: 'simulated bank' when mode is none or rehearsal, 'RBIH Phase-2 public data' only when mode is real. The page must never claim more than the badge says.

14. The 3-minute answer for the judge

Normal-traffic patterns, thresholds and mule behaviour come from the RBIH Phase-2 challenge data, installed through a verified, audited bundle. We simulate the attacks because no dataset contains attacks that can be replayed safely, and we say so. Every decision returns a plain-language reason mapped to RBI guidance with an honest strength label. A bank can plug in its own transaction logs and re-derive the thresholds. Nothing is claimed as compliance; it is design alignment.